

The Impact of Convex Free-Space Decomposition on Graphs of Convex Sets for Motion Planning

First Author¹[0000–1111–2222–3333], Second Author^{2,3}[1111–2222–3333–4444], and
Third Author³[2222–3333–4444–5555]

¹ Princeton University, Princeton NJ 08544, USA

² Springer Heidelberg, Tiergartenstr. 17, 69121 Heidelberg, Germany
lncs@springer.com

<http://www.springer.com/gp/computer-science/lncs>

³ ABC Institute, Rupert-Karls-University Heidelberg, Heidelberg, Germany
{abc,lncs}@uni-heidelberg.de

Abstract. Graphs of Convex Sets (GCS) with Bézier trajectories enable collision-free motion planning through convex optimization, but their practical efficiency depends on the free-space cover. This study compares Approximate Convex Decomposition (ACD) and Visibility Clique Cover (VCC) as front ends to a common GCS–Bézier pipeline. We evaluate cover-construction cost, graph and model size, planning time, reliability, and trajectory quality across five planar map families. ACD solved all 25 test instances and was faster end-to-end under the tested single-query configuration. VCC solved 23 instances; its failures occurred in narrow passages. Where visibility-based grouping reduced graph connectivity, VCC substantially lowered downstream GCS solve time—from 40.10 s to 3.10 s on the **clustering** maps—and achieved higher median minimum clearance in each successful family. However, its computational benefit and reliability did not transfer uniformly across environments, and neither method consistently dominated path length or traversal time. These results indicate that induced edge structure is more informative than region count alone for assessing GCS difficulty. Within this benchmark, ACD is a robust, low-overhead choice for single-query planning, while VCC is most useful when it meaningfully reduces connectivity.

Keywords: Motion planning · Graphs of Convex Sets · Bézier trajectories · convex free-space decomposition · visibility clique cover

1 Introduction

Motion planning requires a robot to reach a goal while remaining collision free and respecting its motion constraints. This task is challenging because obstacles make the free configuration space non-convex. Sampling-based planners, such as probabilistic roadmaps and rapidly exploring random trees, can explore complex spaces effectively [2,3]; trajectory-optimization methods can generate smooth

motions but commonly depend on a feasible initialization and may converge to a local solution [9,7]. Methods that expose convex structure in the free space offer a complementary route to handling this geometric non-convexity.

Graphs of Convex Sets (GCS) formulate motion planning as a joint discrete and continuous optimization problem [6]. A collision-free free-space cover is represented by convex regions and their feasible transitions; GCS then selects a route through this graph while optimizing the trajectory within the chosen regions. When combined with Bézier curves, the convex-hull property provides continuous-time collision guarantees from convex containment constraints [5]. The quality of the convex cover is therefore central to practical performance. It controls not only the number of regions, but also their overlap pattern, the number of GCS edges, and ultimately the size of the optimization problem.

This paper studies that front-end design choice by comparing Approximate Convex Decomposition (ACD) [4] and Visibility Clique Cover (VCC) [8] in the same GCS–Bézier planning pipeline. ACD uses local concavity to decompose polygonal free space, whereas VCC uses sampled visibility and convex-region inflation to seek a compact cover. These methods embody different trade-offs: ACD can be inexpensive to construct but may yield many local regions, while VCC can reduce the downstream graph but incurs sampling, visibility-testing, and region-inflation costs and can be sensitive to narrow passages.

The paper makes the following empirical contributions:

- It provides a paired comparison of ACD and VCC on five two-dimensional map families, using five randomized instances per family and a common GCS–Bézier planner.
- It evaluates decomposition time, GCS graph and presolved-model size, solve time, end-to-end time, success rate, and trajectory quality rather than using the number of regions as the sole proxy for performance.
- It shows that the induced edge structure is a stronger predictor of GCS solve time than region count alone. In the tested single-query setting, ACD is the more reliable and faster end-to-end default, while VCC is most useful when its decomposition can be amortized across repeated queries or when it substantially reduces graph connectivity.

The remainder of the paper is organized as follows. Section 2 summarizes the GCS free-space representation and the two decomposition methods. Section 3 presents the GCS–Bézier planning framework. Section 4 reports the experimental evaluation, and Section 5 concludes the paper and outlines future work.

2 Technical Background

2.1 Convex Free-Space Covers and Graphs of Convex Sets

Obstacle avoidance is difficult because the free configuration space $\mathcal{C}_{\text{free}}$ is generally non-convex. The GCS–Bézier approach represents the portion relevant to a planning query by a finite cover of collision-free convex regions,

$$\mathcal{Q} = \{Q_i\}_{i=1}^{N_R}, \quad Q_i \subseteq \mathcal{C}_{\text{free}}, \quad N_R = |\mathcal{Q}|. \quad (1)$$

For each query, the cover includes designated regions $Q_\sigma, Q_\tau \in \mathcal{Q}$ such that $\mathbf{x}_{\text{start}} \in Q_\sigma$ and $\mathbf{x}_{\text{goal}} \in Q_\tau$.

The cover induces a directed Graph of Convex Sets (GCS), $G = (V, E)$. Each vertex $v_i \in V$ represents Q_i , so $|V| = N_R$, and each directed edge represents a feasible transition between two regions. We write $N_E = |E|$ for the number of such transitions. In particular, an edge is considered only when the two regions have a nonempty common interface and the associated trajectory-compatibility constraints can be satisfied. GCS then jointly chooses a route from σ to τ and the continuous trajectory variables in the selected regions [6,5].

Convexity is essential: every convex combination of points in Q_i remains collision free. Moreover, a nonempty overlap or shared boundary between neighboring regions provides a candidate connection set for consecutive trajectory segments. At the same time, a region that overlaps many others can create many graph edges. Consequently, a cover with fewer regions need not produce a smaller or easier GCS problem; both N_R and N_E affect the downstream model size.

In the GCS-Bézier formulation, a local trajectory is represented by Bézier control points constrained to its assigned region. The convex-hull property of Bézier curves then certifies collision freedom over the whole segment rather than only at sampled points. The following subsections describe the two cover-construction methods compared in this study.

2.2 Approximate Convex Decomposition

Approximate Convex Decomposition (ACD) is a geometry-driven method for decomposing a non-convex polygon [4]. In a standard hull-based construction, concavity is measured by how far a concave boundary feature lies from the polygon’s convex hull. ACD selects a prominent reflex notch, inserts an interior cut that resolves the notch, and recursively repeats this operation. It stops when the residual concavity of every component is below a user-selected tolerance. Thus, a smaller tolerance retains finer geometric features and generally yields more local pieces, whereas a larger tolerance suppresses minor features.

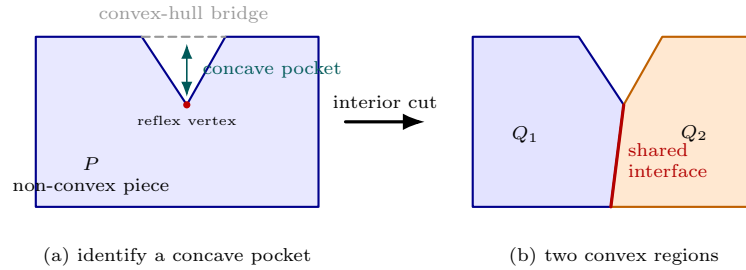


Fig. 1. Conceptual ACD step. A convex-hull bridge exposes a concave pocket; an interior cut from its selected reflex vertex produces two convex regions with a shared interface.

The cut-induced pieces meet along shared boundary portions. When these interfaces admit the GCS compatibility constraints, they become edges in the region graph. Therefore, resolving fine features can create numerous local regions and shared interfaces, increasing N_R and often N_E . ACD is a useful low-cost baseline for testing whether a fast cover-construction stage offsets the optimization cost of a larger downstream graph.

2.3 Visibility Clique Cover

Visibility Clique Cover (VCC) constructs a cover from the global visibility structure of free space [8]. It samples collision-free configurations and connects two samples when their straight-line segment remains collision free. A clique identifies mutually visible samples that initialize a collision-free seed region for inflation. Starting from an ellipsoidal seed, an IRIS-style region-inflation procedure alternates between computing obstacle-separating hyperplanes and expanding a maximum-volume ellipsoid inside their polytope intersection. The converged polytope is a convex, collision-free region [1].

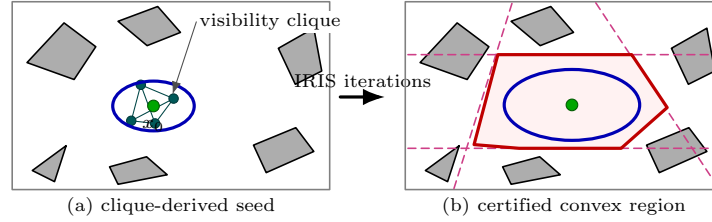


Fig. 2. Conceptual VCC and IRIS-style inflation. Teal points and edges form a visibility clique; its seed (green) is inflated by alternating obstacle-separating hyperplanes (magenta dashed) and maximum-volume ellipsoid updates (blue), yielding the collision-free convex polytope (red).

The resulting polytopes can be larger than local ACD pieces and may overlap. Convexity gives the optimizer freedom to place a Bézier segment anywhere within a region, while an overlap supplies a nonempty connection set for adjacent segments. This can reduce the number of active regions and alter the shape of the optimized path. However, extensive overlap can also create many candidate GCS transitions, so a compact cover does not guarantee a sparse region graph. The total construction cost includes sampling and visibility tests as well as repeated region-inflation calls; it can increase with the sampling budget and coverage target. Sparse or unrepresentative samples can make narrow passages difficult to cover or connect.

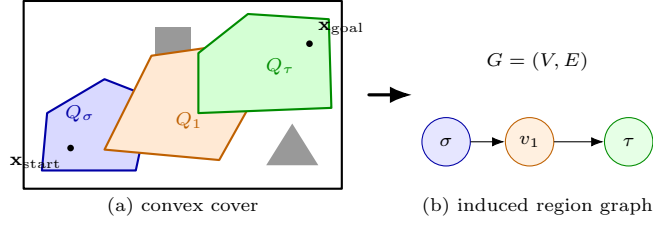


Fig. 3. From a convex free-space cover to a GCS. Each cover region maps to one graph vertex, and each feasible common interface maps to a directed transition. The start and goal lie in the designated source and target regions.

3 Integrated GCS-Bézier Framework for Motion Planning

3.1 Region Graph and Bézier Trajectory Model

The planner starts from the collision-free convex cover $\mathcal{Q} = \{Q_i\}_{i=1}^{N_R}$ in Eq. (1), constructed by either ACD or VCC. Each region Q_i is represented by a graph vertex v_i ; we write $Q_{v_i} = Q_i$ when indexing regions by vertices. Thus $|V| = N_R$ and $|E| = N_E$. The source σ and target τ are the designated vertices associated with the regions containing $\mathbf{x}_{\text{start}}$ and $\mathbf{x}_{\text{goal}}$, respectively. A directed edge $e = (u, v)$ represents a feasible transition from Q_u to Q_v . The discrete problem selects an σ - τ path, while the continuous problem determines a trajectory in its selected regions.

For a region $Q_v \subset \mathbb{R}^{n_c}$, the local trajectory is an order- d Bézier curve

$$\mathbf{q}_v(s) = \sum_{k=0}^d B_{k,d}(s) \mathbf{r}_{v,k}, \quad s \in [0, 1], \quad (2)$$

where $\mathbf{r}_{v,k} \in \mathbb{R}^{n_c}$ are spatial control points and $B_{k,d}$ are the Bernstein basis functions. A second Bézier curve $t_v(s) = \sum_{k=0}^d B_{k,d}(s) h_{v,k}$ parametrizes time. The local decision variable is consequently

$$x_v = (\mathbf{r}_{v,0}, \dots, \mathbf{r}_{v,d}, h_{v,0}, \dots, h_{v,d}).$$

The convex-hull property is the key reason for this representation: if every control point belongs to the convex, collision-free region Q_v , the complete curve belongs to Q_v . In particular, when $Q_v = \{\mathbf{q} \mid A_v \mathbf{q} \leq b_v\}$, continuous-time collision avoidance is certified by the finite set of linear constraints

$$A_v \mathbf{r}_{v,k} \leq b_v, \quad k = 0, \dots, d. \quad (3)$$

This converts continuous-time collision avoidance into finitely many convex constraints [5]. The timing and velocity constraints are written in convex perspective form:

$$\Delta h_{v,k} \geq \varepsilon, \quad \Delta \mathbf{r}_{v,k} \in \Delta h_{v,k} D, \quad k = 0, \dots, d-1, \quad (4)$$

where $\Delta \mathbf{r}_{v,k} = \mathbf{r}_{v,k+1} - \mathbf{r}_{v,k}$, $\Delta h_{v,k} = h_{v,k+1} - h_{v,k}$, and $\varepsilon > 0$ prevents a degenerate time scaling. Here $D \subseteq \mathbb{R}^{n_c}$ is a prescribed convex set of admissible configuration velocities, whose particular parametrization is set by the planning model. For each selected edge (u, v) , position and time are joined through

$$\mathbf{r}_{u,d} = \mathbf{r}_{v,0}, \quad h_{u,d} = h_{v,0}. \quad (5)$$

Together with Eq. (3), this places the shared endpoint in $Q_u \cap Q_v$. Higher-order continuity constraints can be included in the edge model when a particular application requires them.

3.2 Perspective GCS Formulation

Let $\mathcal{X}_v$ collect the local constraints in Eqs. (3)–(4), and let $\mathcal{X}_e$ collect the compatibility constraints for edge $e = (u, v)$. For the designated source and target regions, $\mathcal{X}_\sigma$ and $\mathcal{X}_\tau$ additionally enforce $\mathbf{r}_{\sigma,0} = \mathbf{x}_{\text{start}}$ and $\mathbf{r}_{\tau,d} = \mathbf{x}_{\text{goal}}$, respectively. GCS uses a binary flow variable y_e to indicate whether an edge is selected. Rather than multiplying y_e by the continuous variables directly, the perspective formulation introduces edge-local copies $z_{e,u}$ and $z_{e,v}$. For a convex edge cost ℓ_e , its closed perspective is

$$\tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e) = y_e \ell_e\left(\frac{z_{e,u}}{y_e}, \frac{z_{e,v}}{y_e}\right) \quad \text{for } y_e > 0, \quad (6)$$

with the usual closed extension at $y_e = 0$. This construction preserves convexity and switches the continuous variables and cost off when an edge is inactive [6].

For a vertex v , let $\mathcal{E}^+(v) = \{e = (v, w) \in E\}$ and $\mathcal{E}^-(v) = \{e = (w, v) \in E\}$ denote its outgoing and incoming edge sets, respectively. For an edge $e = (u, v)$, $z_{e,u}$ is the copy of the local trajectory variable at its tail u , and $z_{e,v}$ is the copy at its head v . The resulting GCS–Bézier problem can be expressed compactly as

$$\begin{aligned} \min_{y,z} \quad & \sum_{e=(u,v) \in E} \tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e) \\ \text{s.t.} \quad & \sum_{e \in \mathcal{E}^+(v)} y_e - \sum_{e \in \mathcal{E}^-(v)} y_e = \delta_v, & \forall v \in V, \\ & \sum_{e \in \mathcal{E}^-(v)} z_{e,v} = \sum_{e \in \mathcal{E}^+(v)} z_{e,u}, & \forall v \in V \setminus \{\sigma, \tau\}, \\ & z_{e,u} \in y_e \mathcal{X}_u, \quad z_{e,v} \in y_e \mathcal{X}_v, \quad (z_{e,u}, z_{e,v}) \in y_e \mathcal{X}_e, & \forall e = (u, v) \in E, \\ & y_e \in \{0, 1\}, & \forall e \in E. \end{aligned} \quad (7)$$

Here, $\delta_\sigma = 1$, $\delta_\tau = -1$, and $\delta_v = 0$ otherwise. The first constraint routes one unit of flow from σ to τ . The second equates the head copy of each incoming edge with the tail copy of each outgoing edge at the same region, thereby enforcing state-flow consistency across local trajectories. Depending on the chosen convex costs and sets, this model is a mixed-integer convex program or a mixed-integer second-order cone program [6,5]. It can combine convex penalties on path length, traversal time, and trajectory smoothness.

3.3 Convex Relaxation and Path Rounding

Solving Eq. (7) to global optimality with branch-and-bound can be expensive when the region graph is large. The scalable alternative used for the benchmark first relaxes the binary constraints to $0 \leq y_e \leq 1$ and solves the resulting convex problem. Its optimum C_{relax} is a lower bound on the optimal mixed-integer cost C_{opt} .

The fractional flow is then converted to a valid discrete path by randomized rounding. At a current vertex u , an admissible outgoing edge is sampled according to

$$\Pr(e \mid u) = \frac{y_e^*}{\sum_{e' \in \mathcal{E}^+(u)} y_{e'}^*}, \quad e \in \mathcal{E}^+(u), \quad (8)$$

while excluding edges that would create a cycle. Once a path reaches τ , its edge variables are fixed and the continuous trajectory variables are re-optimized over that convex restriction. The resulting feasible cost C_{round} satisfies

$$C_{\text{relax}} \leq C_{\text{opt}} \leq C_{\text{round}}. \quad (9)$$

Accordingly, the reported relaxation–rounding gap

$$g_{\text{relax}} = \frac{C_{\text{round}} - C_{\text{relax}}}{C_{\text{relax}}} \quad (10)$$

is a computable upper bound on the unknown relative suboptimality whenever $C_{\text{relax}} > 0$. A zero gap certifies optimality of the rounded solution; otherwise, branch-and-bound is required for an exact global certificate. This distinction motivates reporting both graph size and relaxation–rounding behavior in the experimental analysis.

4 Experimental Results and Discussion

4.1 Benchmark Overview

The benchmark evaluates two automatic free-space decomposition methods, Approximate Convex Decomposition (ACD) and Visibility Clique Cover (VCC), inside the same GCS–Bézier motion-planning pipeline. Five map families are used: **clustering**, **narrows**, **rooms**, **u_shaped**, and **flappy**. Each family is generated with five seeds under the same workspace scale, obstacle density, obstacle size, and start–goal configuration, giving 50 paired runs in total.

ACD solved all 25 instances. VCC solved 23 out of 25 instances; both failures occurred on the **narrows** family, at seeds 2 and 3. The failed VCC diagnostics still reported moderate graph sizes before termination: 11 vertices and 26 edges for seed 2, and 15 vertices and 57 edges for seed 3. Thus, failure is not explained only by graph size. The more likely cause is the interaction between visibility sampling, coverage requirements, start–goal connectivity, and the narrow-gateway geometry.

4.2 Model Size and Runtime

Table 1 summarizes the main model-size and timing metrics. Each entry is the median over successful seeds. For VCC on **narrows**, the statistics are computed over the three successful runs, while the success rate remains reported as 3/5.

Table 1. Median model size and runtime by map family. Time is measured in seconds.

Map	Method	Success	N_R	N_E	t_{dec}	t_{GCS}	$t_{\text{dec}} + t_{\text{GCS}}$
Clustering	ACD	5/5	104	462	7.73	40.10	59.30
	VCC	5/5	22	99	85.80	3.10	88.80
Narrows	ACD	5/5	13	40	0.68	0.83	1.52
	VCC	3/5	11	50	13.10	0.73	13.80
Rooms	ACD	5/5	15	38	0.78	0.50	1.28
	VCC	5/5	13	57	15.30	1.33	18.90
U-shaped	ACD	5/5	25	86	1.29	2.77	4.07
	VCC	5/5	16	89	16.30	1.93	18.30
Flappy	ACD	5/5	13	26	0.98	0.16	1.13
	VCC	5/5	13	28	12.80	0.16	12.90

VCC is most effective on the **clustering** maps. It reduces the median number of regions from 104 to 22, the number of GCS edges from 462 to 99, and the GCS wall time from 40.10 s to 3.10 s. This confirms that visibility-based grouping can substantially reduce the downstream optimization burden when the free space contains many locally redundant convex pieces.

The same conclusion does not hold uniformly across all map families. On **u_shaped**, VCC reduces the number of regions from 25 to 16, but the median edge count is essentially unchanged. On **rooms**, VCC has an outlier with 33 regions, 430 edges, and a GCS wall time of 105 s. These cases show that the edge structure of the GCS graph matters as much as the number of convex regions.

When decomposition time is included, ACD is faster in all five map families under the tested configuration. The VCC setup uses 500 samples and a target coverage of 0.96, making its decomposition cost much larger than that of ACD. Even for **clustering**, where VCC reduces GCS solve time by roughly one order of magnitude, its median decomposition time of 85.80 s makes the end-to-end runtime higher than ACD. VCC is therefore more attractive when the same decomposition can be reused for many planning queries; ACD is preferable for one-shot planning in this benchmark.

4.3 Correlation Between Model Size and Solve Time

Figure 4 shows the relationship between model-size indicators and solve time over the 48 successful runs. The strongest Spearman correlations with **SolvePath** time are obtained by the presolved model size: $\rho = 0.986$ for the number of presolved variables and $\rho = 0.983$ for the number of presolved constraints. Among graph-level quantities available before solver construction, the number of edges

is more predictive than the number of vertices, with $\rho = 0.982$ compared with $\rho = 0.839$.

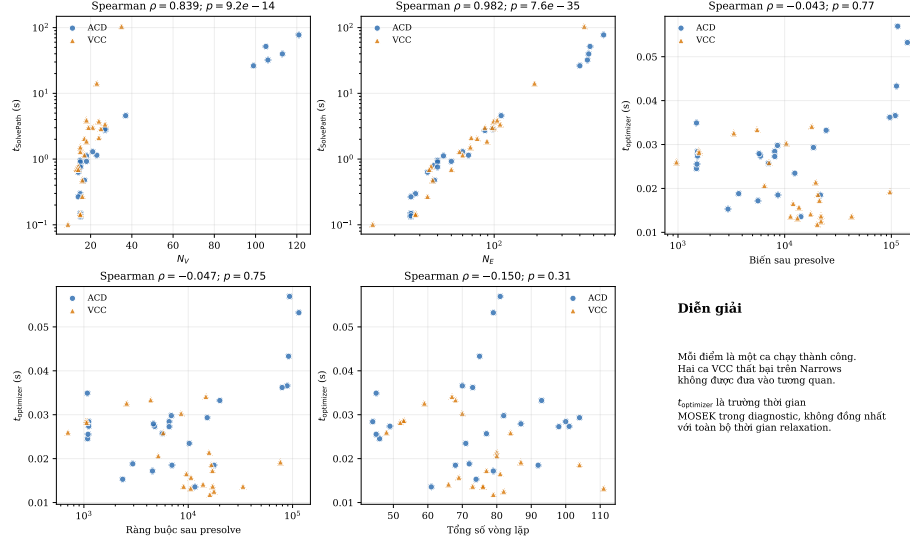


Fig. 4. Relationships between model-size indicators and solve time over 48 successful runs. Log axes are used where the metric range spans several orders of magnitude.

This result is consistent with the GCS formulation. Edges do not merely encode topological adjacency; they also introduce flow variables and continuity constraints between adjacent convex sets. A decomposition with fewer regions can still create a difficult optimization problem if it induces a dense graph. Consequently, the number of regions alone is an incomplete proxy for computational difficulty.

4.4 Trajectory Quality

Table 2 reports median path quality metrics. Neither method dominates in path length or trajectory time. ACD produces shorter paths on **clustering** and **u_shaped**; VCC performs better on **rooms**, is nearly equivalent on **flappy**, and has only three successful **narrows** runs for comparison.

VCC gives a higher median minimum clearance in every successful map family. This suggests that visibility-based regions often keep the rounded trajectory farther from obstacles. However, this safety margin is not free: it comes with higher decomposition cost and lower robustness on narrow passages. The relaxation and rounding objectives also vary by topology. On **flappy**, the relaxation and rounded objectives are nearly identical, indicating a tight relaxation for

Table 2. Median trajectory-quality metrics over successful runs.

Map	Method	Path length	Traj. time	Min. clearance	Active regions
Clustering	ACD	7.09	8.82	6.38×10^{-5}	13
	VCC	8.29	8.38	1.66×10^{-1}	5
Narrows	ACD	12.50	15.70	4.79×10^{-4}	9
	VCC	12.00	16.10	2.50×10^{-3}	7
Rooms	ACD	8.19	8.02	5.71×10^{-3}	7
	VCC	7.63	7.40	1.24×10^{-2}	8
U-shaped	ACD	7.47	7.26	5.36×10^{-4}	8
	VCC	7.95	7.96	4.61×10^{-2}	5
Flappy	ACD	21.40	26.30	2.98×10^{-4}	11
	VCC	21.30	25.40	5.64×10^{-4}	13

this corridor-like structure. On **clustering**, VCC exhibits a larger relaxation-rounding gap, showing that a compact continuous relaxation does not always preserve solution quality after discrete path selection.

4.5 Visual Evidence on Flappy Maps

The **flappy** family provides a useful qualitative check because each instance forces a winding route through alternating upper and lower openings. Figure 5 compares ACD and VCC on all five **flappy** seeds. Both methods find feasible trajectories in every seed, and their path-quality metrics are close. The main difference is computational: the median end-to-end runtime is 1.13 s for ACD and 12.90 s for VCC, mostly because VCC decomposition dominates the pipeline.

4.6 Discussion and Limitations

The main conclusion is that region count alone is not sufficient to characterize GCS difficulty. A useful decomposition must be evaluated through the induced edge count, presolved model size, relaxation and rounding behavior, decomposition time, and final trajectory quality. Under the tested one-query pipeline, ACD is the more reliable and faster default. VCC becomes compelling when the decomposition can be amortized across repeated solves or when the environment resembles **clustering**, where the visibility graph substantially reduces the GCS solve time.

The study has several limitations. Each map family contains only five seeds, and VCC has only three successful **narrows** runs. All experiments use the same workspace scale, obstacle density, obstacle size, and area-coverage mode. The reported values are extracted from diagnostic outputs; metrics not shown in those diagnostics, such as mean clearance and smoothness proxies, are not analyzed. A broader evaluation should vary workspace scale, obstacle density, VCC sampling budget, coverage target, and the number of seeds.

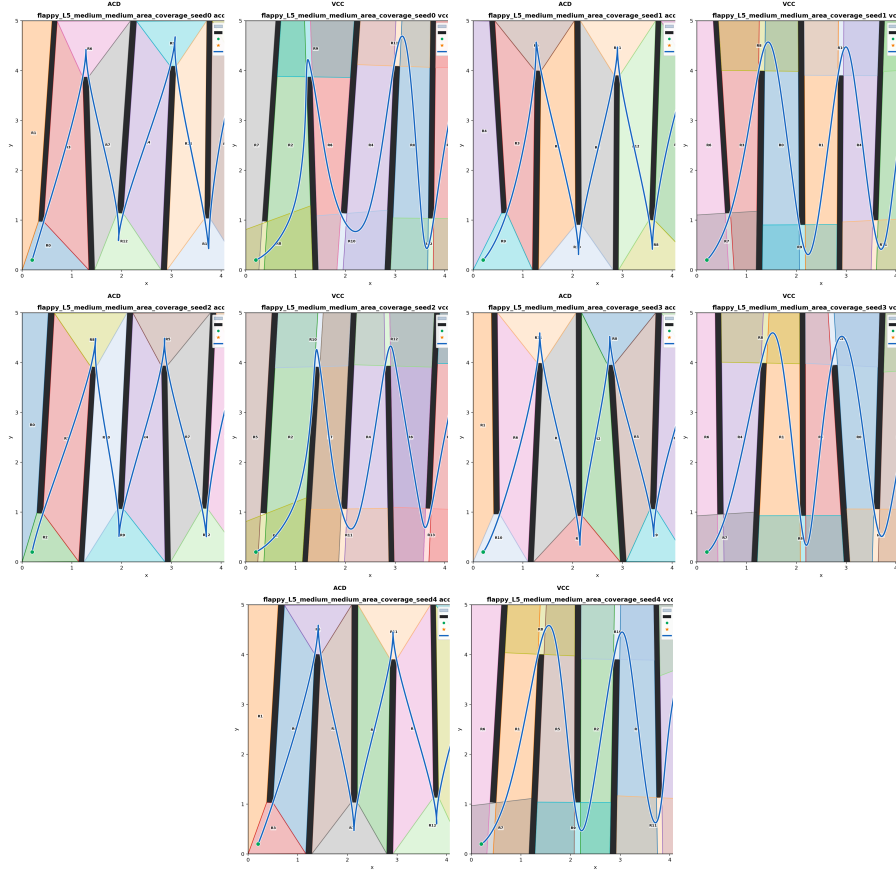


Fig. 5. ACD and VCC comparisons on the five **flappy** seeds. Obstacles are black, convex regions are colored, and the GCS trajectory is plotted in blue.

5 Conclusion and Future Work

This paper compared Approximate Convex Decomposition (ACD) and Visibility Clique Cover (VCC) as front ends for a common GCS–Bézier motion-planning pipeline. The experiments show that the usefulness of a convex cover cannot be judged from its region count alone. Although VCC substantially reduced the number of regions and GCS edges on the `clustering` maps, comparable reductions did not occur uniformly across the other environments. Over the 48 successful runs, edge count was a stronger graph-level predictor of solve time than region count. In the tested single-query configuration, ACD solved all 25 instances and was faster end-to-end in every map family because its decomposition cost was much lower. VCC solved 23 instances, produced higher median minimum clearance in every successful family, and could substantially reduce downstream GCS time when visibility-based grouping eliminated redundant connectivity. Thus, the relevant design trade-off is between total decomposition-and-planning cost, the connectivity induced by the cover, and the desired trajectory properties, rather than compactness of the cover by itself.

Future work should test these findings on a larger and more varied benchmark, including additional seeds, workspace scales, obstacle densities, narrow passages, higher-dimensional configuration spaces, and robot geometries. An important next step is to evaluate repeated-query settings, where the one-time cost of a VCC cover may be amortized, and to study adaptive sampling, coverage, and region-inflation policies that explicitly control the resulting GCS connectivity. It would also be useful to report richer trajectory measures—such as clearance distributions, smoothness, and dynamic feasibility—and to compare different rounding strategies or exact solvers when the relaxation–rounding gap is large. Together, these extensions could support a principled, problem-dependent choice of free-space decomposition for GCS motion planning.

Acknowledgments. We acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for supporting this study.

References

1. Deits, R., Tedrake, R.: Computing large convex regions of obstacle-free space through semidefinite programming. In: Algorithmic Foundations of Robotics XI, Springer Tracts in Advanced Robotics, vol. 107. Springer (2015). https://doi.org/10.1007/978-3-319-16595-0_7, https://groups.csail.mit.edu/robotics-center/public_papers/Deits14.pdf
2. Kavraki, L.E., Svestka, P., Latombe, J.C., Overmars, M.H.: Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation* **12**(4), 566–580 (1996). <https://doi.org/10.1109/70.508439>
3. LaValle, S.M.: Rapidly-exploring random trees: A new tool for path planning. Tech. Rep. TR 98-11, Iowa State University (1998)

4. Lien, J.M., Amato, N.M.: Approximate convex decomposition of polygons. *Computational Geometry: Theory and Applications* **35**(1–2), 100–123 (2006). <https://doi.org/10.1016/j.comgeo.2005.10.005>
5. Marcucci, T., Petersen, M., von Wrangell, D., Tedrake, R.: Motion planning around obstacles with convex optimization. *Science Robotics* **8**(84) (2023). <https://doi.org/10.1126/scirobotics.adf7843>, <https://arxiv.org/abs/2205.04422>
6. Marcucci, T., Umenberger, J., Parrilo, P.A., Tedrake, R.: Shortest paths in graphs of convex sets. *SIAM Journal on Optimization* **34**(1), 507–532 (2024). <https://doi.org/10.1137/22M1523790>, <https://arxiv.org/abs/2101.11565>
7. Schulman, J., Duan, Y., Ho, J., Lee, A., Awwal, I., Bradlow, H., Pan, J., Patil, S., Goldberg, K., Abbeel, P.: Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research* **33**(9), 1251–1270 (2014). <https://doi.org/10.1177/0278364914528132>
8. Werner, P., Amice, A., Marcucci, T., Rus, D., Tedrake, R.: Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs. In: 2024 IEEE International Conference on Robotics and Automation (ICRA). pp. 10359–10365 (2024). <https://doi.org/10.1109/ICRA57147.2024.10610005>, <https://arxiv.org/abs/2310.02875>
9. Zucker, M., Ratliff, N., Dragan, A.D., Pivtoraiko, M., Klingensmith, M., Dellin, C.M., Bagnell, J.A., Srinivasa, S.S.: CHOMP: Covariant Hamiltonian optimization for motion planning. *The International Journal of Robotics Research* **32**(9–10), 1164–1193 (2013). <https://doi.org/10.1177/0278364913488805>